

Scalable Fine-tuning from Multiple Data Sources: A First-Order Approximation Approach

Dongyue Li^{1*}, Ziniu Zhang^{1*}, Lu Wang², and Hongyang R. Zhang¹

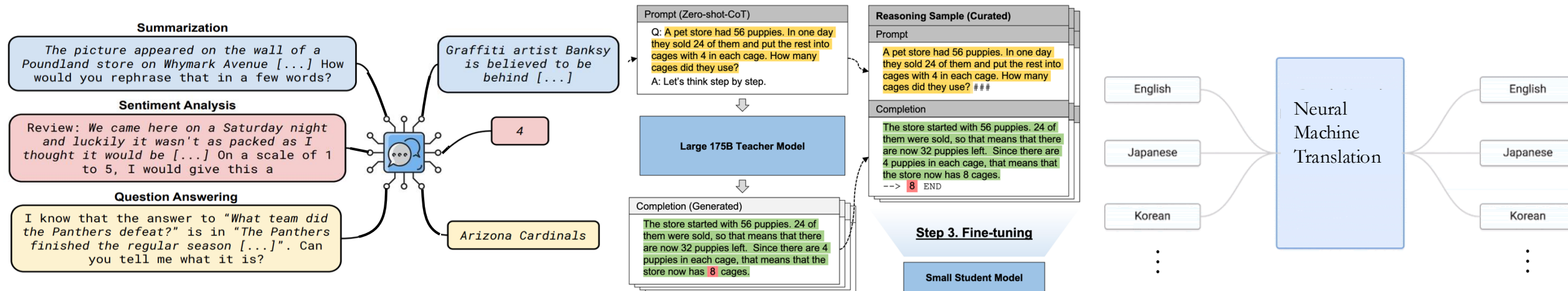
¹Northeastern University, Boston, MA

²University of Michigan, Ann Arbor, MI



Language Model Fine-tuning

- In many applications, fine-tuning language models involves many data sources.
 - Instruction tuning, such as FLAN v2 (with 1800+ tasks) (Chung et al., '22)
 - Fine-tuning with LLM-generated contents, e.g., chain-of-thoughts (Ho et al., '23), contexts generated for open-domain QA (Zhang et al., '23)
 - Training multilingual models, e.g., neural machine translation (Neubig and Hu, '18) and code generation for multiple PLs (Cassano et al. '22)



Data Subset Selection for Fine-tuning LMs

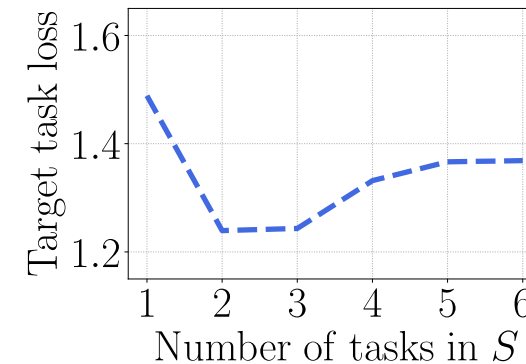
- A fundamental problem in fine-tuning LM is selecting the most beneficial data sources among all available data sources:
 - Suppose we have access to n tasks. Denote S as a subset of $\{1, \dots, n\}$.
 - Let $f(S)$ be the fine-tuned model loss evaluated on the target task.
- **Examples.** Fine-tuning a Llama-1B model on the Alpaca dataset (split into 38 tasks by instruction types).
 - Combination of tasks can have a negative effect on $f(S)$:

For 47% pairwise combinations of tasks $f(\{i, j\})$ on average, it dips below the performance of fine-tuning on a single task $f(\{i\})$.



Data Subset Selection for Fine-tuning LMs

- A fundamental problem in fine-tuning LM is selecting the most beneficial data sources among all available data sources:
 - Suppose we have access to n tasks. Denote S as a subset of $\{1, \dots, n\}$.
 - Let $f(S)$ be the fine-tuned model loss evaluated on the target task.
- **Examples.** Fine-tuning a Llama model on the Alpaca dataset (split into 38 tasks by instruction types).
 - $f(S)$ is not monotone: gradually adding “beneficial” task j to $S = \{i\}$ has diminishing effects or even decreases $f(S)$



Data Subset Selection for Fine-tuning LMs

- A fundamental problem in fine-tuning LM is selecting the most beneficial data sources among all available data sources:
 - Suppose we have access to n tasks. Denote S as a subset of $\{1, \dots, n\}$.
 - Let $f(S)$ be the fine-tuned model loss evaluated on the target task.
- **Examples.** Fine-tuning a Llama model on the Alpaca dataset (split into 38 tasks by instruction types).
 - Gradient and representation-based similarity scores weakly correlate with $f(S)$: both yield a Spearman correlation score less than 0.2.



Data Subset Selection for Fine-tuning LMs

- A fundamental problem in fine-tuning LM is selecting the most beneficial data sources among all available data sources:
 - Suppose we have access to n tasks. Denote S as a subset of $\{1, \dots, n\}$.
 - Let $f(S)$ be the fine-tuned model loss evaluated on the target task.
- **Examples.** Fine-tuning a Llama model on the Alpaca dataset (split into 38 tasks by instruction types).
 - Combination of tasks can have a negative effect on $f(S)$.
 - $f(S)$ is not monotone.
 - Gradient and representation-based similarity scores weakly correlate with $f(S)$.

It is desirable to build a method that selects subsets directly based on $f(S)$.



Classic Subset Selection Algorithms

- Classical subset selection methods have been effective for feature selection.
- **Forward stepwise selection** (Hastie et al., '09).
 - Enumerate the tasks that have not yet been added to the current subset S' .
 - Add the task j that leads to best $f(S)$ after combining j to S' .
 - Repeat until no task can be further added to S .
- **Random ensemble** (Ilyas et al., '22; Li et al., '23).
 - Sample m random subsets from $\{1, 2, \dots, n\}$: $S_1, \dots, S_m$.
 - Fit one model on each subset of tasks.
 - Compute a score for task j as the average $f(S)$ among subsets that contain j .



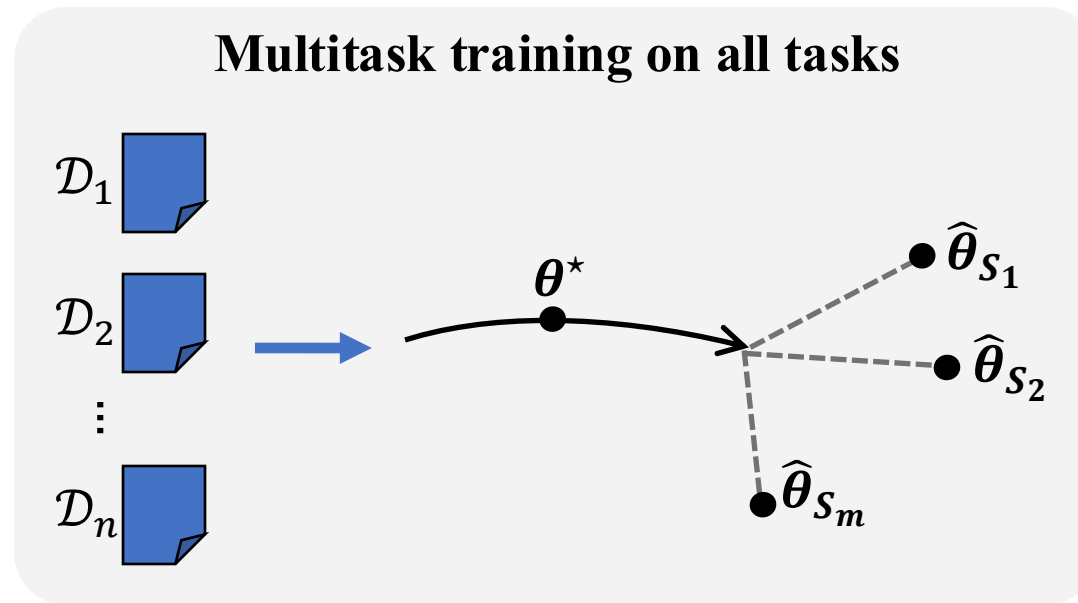
Problem Statement

- Applying the classic subset selection methods requires **repeatedly fine-tuning language models on numerous subsets**, which is computationally expensive.
 - **Forward stepwise selection:** $O(n^2)$ subsets, including $\{1\}, \{2\}, \dots \{1, 2\}, \{1, 3\}, \dots, \{1, \dots, n\}$.
 - **Random ensemble:** $\Omega(n \log n)$ subsets where each one is a random subset.
 - For the Alpaca dataset with 38 tasks, both procedures need to fine-tune models on 700+ and 380 subsets, respectively.
- Given a list of task subsets, can we quickly estimate the fine-tuning model loss $f(S)$, **without repeated fine-tuning language models on each subset?**



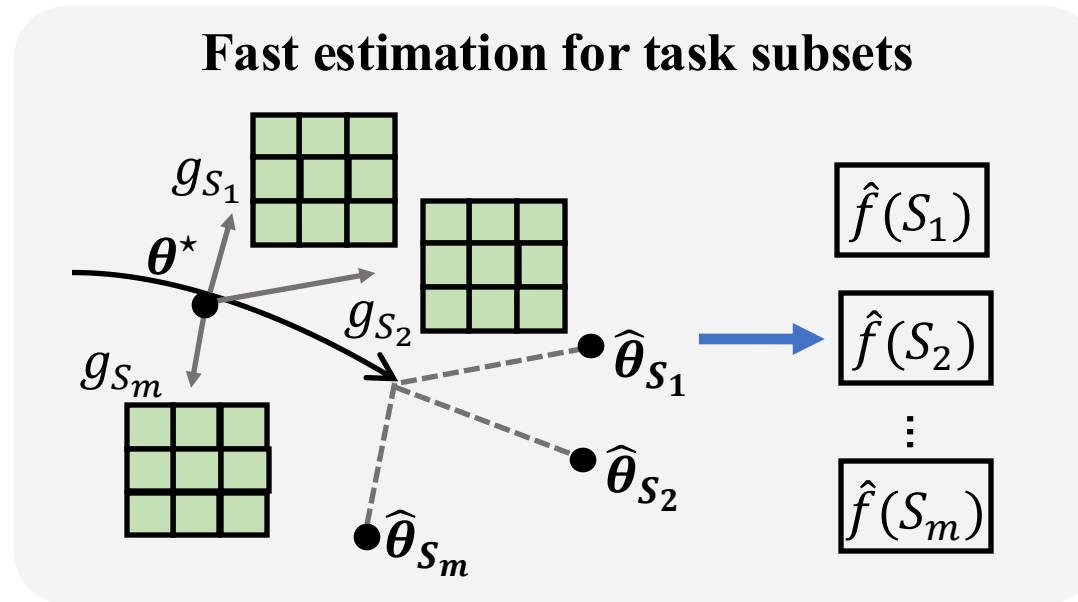
Estimation of Fine-Tuning Losses Using Gradients

- **Our key idea** is to circumvent repeated fine-tuning with a regression-based estimation of fine-tuned model parameters on a particular subset of tasks.
 - **First**, train a meta-initialization by multitask learning on all tasks, denoted as θ^* .
 - Similar to MAML (Finn et al., ICML'17), this initialization can be quickly adapted to a subset of tasks.



Estimation of Fine-Tuning Losses Using Gradients

- **Our key idea** is to circumvent repeated fine-tuning with a regression-based estimation of fine-tuned model parameters on a particular subset of tasks.
 - **First**, train a meta-initialization by multitask learning on all tasks, denoted as θ^* .
 - **Second**, estimate the model fine-tuning loss on a subset of tasks by solving a logistic regression problem with the gradients at θ^* and the task labels.



First-Order Approximation of Fine-tuned Models

- **Key observation.** When the fine-tuned model parameter X is close to θ^* , the model output $h_X(s, y)$ can be approximated by a first-order Taylor's expansion centered at the meta-initialization θ^* :

$$h_X(s, y) \approx h_{\theta^*}(s, y) + \nabla_X h_{\theta^*}(s, y)^\top (X - \theta^*) + \epsilon$$

- We empirically measure ϵ across 12 language models. The results are averaged over 50 random task subsets.
 - The Relative RSS is less than 1% when X is within 10% relative distance to θ^* .

Model	Pythia-70M	BERT-Base	RoBERTa-Base	GPT-2
RRSS	0.7%	0.5%	0.5%	0.5%
Model	FLAN-T5-Base	BloomZ-560M	TinyLlama-1.1B	GPT-Neo-1.3B
RRSS	0.5%	0.5%	0.5%	0.5%
Model	OPT-1.3B	Gemma-2-2B	Mistral-7B	Llama-3-8B
RRSS	0.5%	0.4%	0.4%	0.1%

$$\text{RRSS} = \frac{||\epsilon||^2}{||h_X(s, y)||^2}$$

Fast Estimation Using Gradients

- **Estimating fine-tuned model parameters:**

- Apply the first-order approximation to the log-loss of the model:

$$\ell(X) = \log(1 + \exp(-y_i h_X(s_i, y_i)))$$

$$\Rightarrow \hat{\ell}(X) = \log(1 + \exp(-y_i g_i^\top (X - \theta^*) - y_i h_{\theta^*}(s_i, y_i))).$$

- Solve logistic regression on a subset of tasks S , using their gradient $g_i = \nabla_X h_{\theta^*}(s_i, y_i)$ as the feature vector and y_i as the response label.

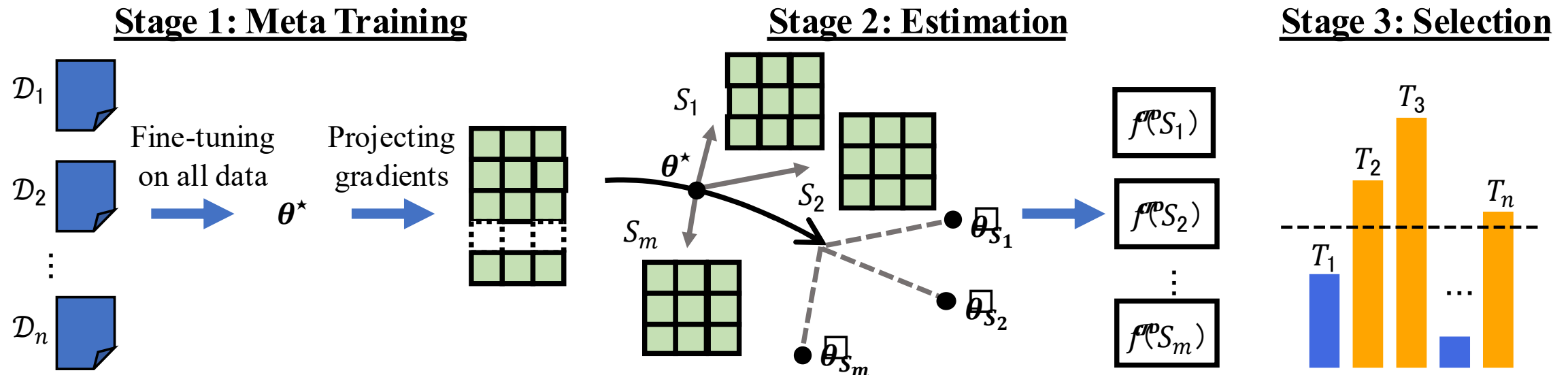
- **Dimension reduction:** Project the gradients to a much lower dimension d without much loss of accuracy, using the Johnson-Lindenstrauss lemma (Johnson, 1984).

Solving logistic regression on CPUs takes within **2 seconds per instance!**



GradEx: Estimation of Fine-Tuning Losses Using Gradients

- GradEx applies **Forward Stepwise Selection (FS)** and **Random Ensemble (RE)**, estimating the fine-tuning model losses of every subset.



- GradEx reduces the $m = \Omega(n)$ model fine-tuning to only 1.
- GradEx trade-off model fine-tuning with $O(n)$ gradient computation on all data samples and solving logistic regression m times.



Results for Approximating Model Fine-tuning Losses

- GradEx approximates the fine-tuning losses within **1%** relative error, while reducing FLOPS by up to **30x** and **44x**, compared to Forward Selection and Random Ensemble.

Alpaca	GPT-2	FLAN-T5-Base	TinyLlama-1.1B	GPT-Neo-1.3B	Llama-3-8B	Speedup
GRADEX-FS	7.4×10^{-4}	3.2×10^{-4}	3.9×10^{-4}	3.5×10^{-4}	2.7×10^{-4}	17.6×
GRADEX-RE	8.7×10^{-4}	3.4×10^{-4}	4.2×10^{-4}	3.8×10^{-4}	2.9×10^{-4}	43.3×
StrategyQA	GPT-2	FLAN-T5-Base	TinyLlama-1.1B	GPT-Neo-1.3B	Llama-3-8B	Speedup
GRADEX-FS	7.4×10^{-3}	3.0×10^{-4}	3.2×10^{-4}	3.0×10^{-4}	2.4×10^{-4}	30.5×
GRADEX-RE	8.9×10^{-3}	3.6×10^{-4}	3.8×10^{-4}	3.4×10^{-4}	2.8×10^{-4}	44.8×

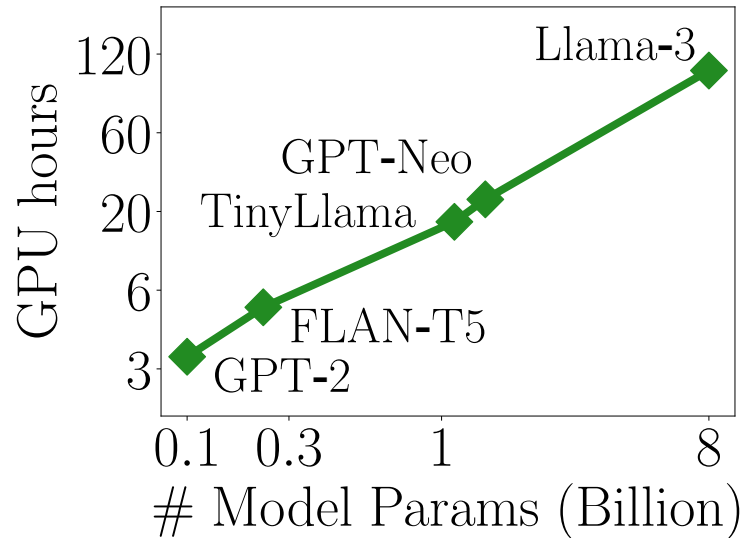
Table. Evaluating the relative error between the estimated and true fine-tuning model losses on both the Alpaca and StrategyQA dataset, using five LMs.

$$\text{Relative error: } \frac{1}{m} \sum_{i=1}^m \frac{(f(S_i) - \hat{f}(S_i))^2}{f(S_i)^2}$$

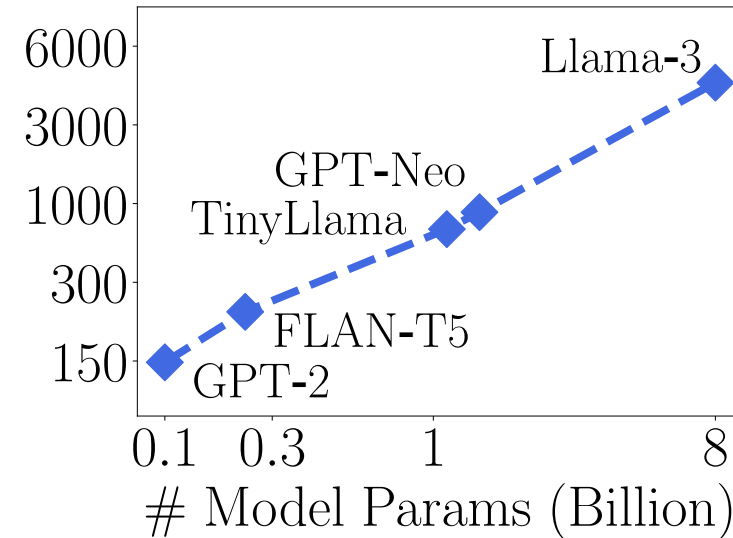


Scaling with the Language Model Size

- The speedup of GradEx remains consistent across different models when applied to the same dataset, reducing the GPU hours by up to **46x**.



GradEx



Full fine-tuning

Figure. The number of GPU hours vs. the size of the language model to compute random ensemble, evaluated on the Alpaca Dataset.



Results for Downstream Fine-tuning Performances

- Our approach delivers comparable test accuracy to classic subset selection methods, incurring less than **0.5%** computation cost.
- Our approach outperforms existing selection methods (DSIR, DEFT, and LESS) by up to **3.8%**, with comparable computational costs.

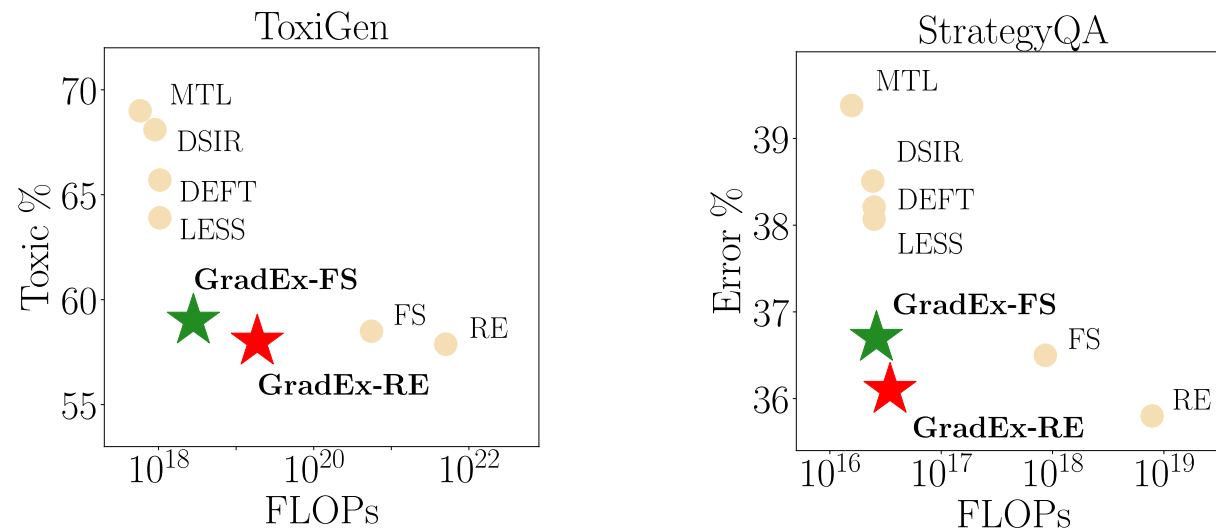


Figure. The tradeoff between error rate and FLOPs

Left: Task selection for instruction tuning

Right: Data selection for chain-of-thought fine-tuning



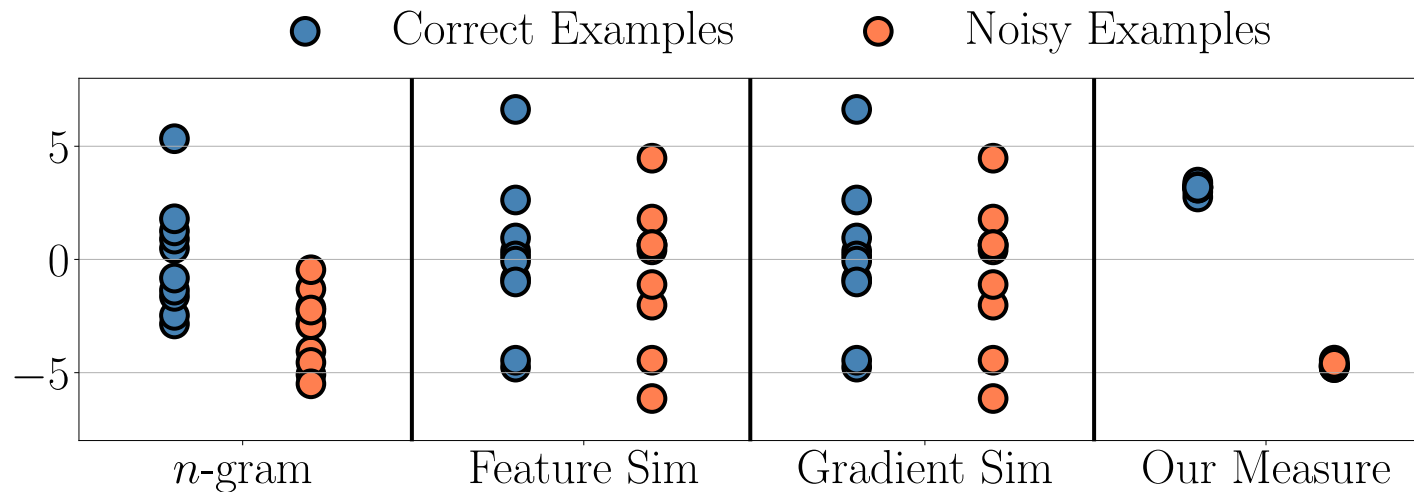
Examples of Our Measures for Task Selection

- **Setup:** A synthetic addition task with noisy examples.

Input: 6 7 0 1 3 + 2 3 9 2 4

Output: 0 7 | 0 3 7 | 0 9 3 7 | 1 0 9 3 7 | 9 0 9 3 7

- The measure in our approach (T) leads to a much clearer separation of correct vs. noisy examples compared to other existing measures.



Summary

- We present an algorithm, GradEx, for quickly estimating the fine-tuning model loss on any given subset of datasets, by trading off repeated fine-tuning with linear regression on gradients.
- GradEx can be applied to classic subset selection algorithms, yielding comparable performance with a significant reduction of computation.
- Our overall algorithm outperforms existing data selection algorithms based on gradient or representation similarities.

Thanks for Listening!

- Paper: <https://arxiv.org/pdf/2409.19458>
- Code: <https://github.com/VirtuosoResearch/Scalable-finetuning>
- Emails: li.dongyu@northeastern.edu, zhang.zini@northeastern.edu, ho.zhang@northeastern.edu, wangluxy@umich.edu

